

Built-in module variants

Intent

`Predict`, `ChainOfThought`, and `ReAct` cover most programs, but DSPy ships a handful of other modules for situations where one LM call is not enough, or where reasoning needs a Python runtime, or where you want to fan out across examples. This page collects those modules, groups them by what they’re for, and gives selection guidance so you know which one to reach for when.

Read this when a plain `Predict` or `ChainOfThought` isn’t getting you there and you’re trying to decide between sampling more, comparing drafts, executing code, or running the same module in parallel.

Design decisions

1. These variants wrap the spine; they don’t replace it

`BestOfN` and `Refine` take a module and sample it. `MultiChainComparison` consumes completions produced by an outer `Predict`. `ProgramOfThought` and `CodeAct` hold internal `ChainOfThought` predictors. The variants are recipes built on the spine, not parallel implementations of it. Before reaching for one, ask whether the question is “I need more of what `Predict` already does” or “I need something `Predict` can’t do at all” — the answer is usually the former, and that’s the case these handle.

2. Sampling-and-aggregating modules vary one thing: the rollout ID

Both `BestOfN` and `Refine` deep-copy the wrapped module per attempt and swap in an LM copy with `rollout_id = start + i` and `temperature=1.0`. The rollout ID is what makes each sample produce a different output despite the same inputs — DSPy threads it into the LM cache key so the model resamples instead of replaying a cached response. Temperature is forced to 1.0 regardless of the LM’s default, because sampling is the point.

3. Reward functions are metrics graded at inference time

The signature `reward_fn(args, pred) -> float` mirrors the inference-time metric shape, but it does not need a gold example — it scores the prediction against the call’s inputs. The same function shape is reusable as a training metric, but a reward is graded *now* and decides which sample to keep, while a metric is graded *later* and decides which program to keep. The two roles overlap in shape and diverge in purpose.

4. MultiChainComparison expects pre-generated completions as its input

`forward(completions, **kwargs)` takes the completions as a positional argument rather than producing them. The caller produces M samples through a separate `Predict` (typically with `n=M`) and hands them in. Splitting sampling from judging keeps the two concerns independent: the completions could come from different LMs, from cache, or from earlier signatures — none of which `MultiChainComparison` needs to know about.

5. ProgramOfThought and CodeAct ship with a Python interpreter

Both rely on `PythonInterpreter`, which runs LM-generated code in a sandbox via Deno’s WASM runtime. Deno isolation is the reason: the LM’s code runs in a process with no filesystem or network access by default, so executing untrusted output is bounded. Installing Deno is a hard dependency for both modules; the constructor raises if Deno isn’t available.

6. CodeAct is ReAct plus a code sandbox

The class literally inherits from both: `class CodeAct(ReAct, ProgramOfThought)`. The combination matters because some tasks need both the iterative loop of `ReAct` (think → act → observe) and the expressive power of writing Python (loops, list comprehensions, library calls). Tools are passed as plain `def` functions and dropped into the sandbox via `inspect.getsource`, so the LM calls them as regular Python rather than as JSON-shaped tool calls.

7. Parallel is a runner, not a Module

It exposes a `forward(exec_pairs)` method but isn’t a `Module` subclass. It doesn’t hold predictors, doesn’t serialize, doesn’t appear in `named_predictors`. The work isn’t “an LM call I’m doing on your behalf” — it’s “a batch of LM calls you’ve assembled, run in parallel.” Keeping it outside the Module tree means optimizers and `dump_state` ignore it, which is what you’d want from a runner.

8. dspy.Parallel and Module.batch overlap on purpose

Both run modules in parallel using the same `ParallelExecutor` underneath. The difference is what they accept: `Module.batch(examples)` takes one module and many examples; `dspy.Parallel()(exec_pairs)` takes many `(module, example)` pairs. Use `Module.batch` for the common case — evaluating one program on a dataset; reach for `Parallel` when the modules differ across examples or you want fine-grained control over which pair runs where.

9. majority is the no-LM aggregator

A plain function — no LM call, no signature, no `dspy.Module`. It tallies completions and returns the most-common normalized value (with `default_normalize` folding case and whitespace). Aggregation should not look like a step that calls a model, so the API is a function call, not a module constructor. Pair it with a multi-sample `Predict` or with `BestOfN`’s `pred.completions` when the task has a discrete answer.

10. RLM is marked experimental for a reason

The class is decorated with `@experimental` and the interface is still in flux. It composes a code sandbox with built-in `llm_query` / `llm_query_batched` tools that let generated code call a separate sub-LM mid-execution. The mental model is a Python REPL the LM drives, with another LM available as a callable inside. Useful, but the boundary conditions — max call counts, sandbox lifetime, error recovery — are still being worked out.

API walkthrough

Grouped by what you’re trying to do.

Sampling and aggregating

For when one LM call has too much variance. Sample several, then pick or combine.

dspy.BestOfN(module, N, reward_fn, threshold, fail_count=None) On `forward(**kwargs)`, `BestOfN` deep-copies the wrapped module per attempt, swaps in a fresh LM with `rollout_id = start + i` and `temperature=1.0`, and runs the call inside a `dspy.context(trace=[])` block so the per-attempt trace is isolated. It scores `reward_fn(kwargs, pred)`, keeps the best so far, and short-circuits when a reward meets `threshold`. After the loop, the winning attempt’s trace is merged back into the parent `dspy.settings.trace` — so a caller inspecting the trace sees the winning path, not all `N`. Failures decrement `fail_count` (defaults to `N`); exhausting it re-raises.

dspy.Refine(module, N, reward_fn, threshold, fail_count=None) Same shape as `BestOfN`, with feedback generation between attempts. After a failed attempt, `Refine` builds a snapshot — the module’s source code, per-predictor signatures, per-predictor I/O from the trace, the reward function’s source — and feeds it to an internal `dspy.Predict(OfferFeedback)`. That call returns advice keyed by module name. On the next attempt, `Refine` wraps the active adapter so each sub-predictor’s signature gains a `hint_` input field carrying its slice of the advice. The wrapping is scoped via `dspy.context(adapter=...)`, so the wrapped adapter exists only for that attempt and the hints don’t leak into the final trace.

dspy.majority(prediction_or_completions, normalize=default_normalize, field=None) A standalone function. Accepts a `Prediction` (it uses `prediction.completions`), a `Completions` object, or a plain list. The `field` defaults to the *last* output field of the signature — the convention being that the last field is the answer. Values pass through `normalize` (lowercase + whitespace by default) and the most-common normalized value wins; ties go to the earlier completion. Returns a single-completion `Prediction` wrapping the winner.

Comparing pre-generated drafts

dspy.MultiChainComparison(signature, M=3, temperature=0.7, **config) The constructor mutates the input signature: it prepends one output field (a synthesized `rationale`) and appends M input fields named `reasoning_attempt_1` through `reasoning_attempt_M`. It then builds an internal `Predict` over the modified signature. At call time, `forward(completions, **kwargs)` reads each completion’s `rationale` (or `reasoning`) and the last output field, formats them as one-line attempt strings, and supplies them as the new inputs. The LM is asked to reason holistically across attempts and produce one synthesized answer in the original output field. The number of supplied completions must equal `M`; an assertion enforces this.

Generating and running code

For tasks where the answer is best computed, not narrated.

dspy.ProgramOfThought(signature, max_iters=3, interpreter=None) Holds three internal `ChainOfThought` predictors: `code_generate` produces Python, `code_regenerate` rewrites it after an execution error, and `generate_output` extracts the declared output fields from the run’s printed result. The forward loop asks `code_generate` for code, runs it through the `PythonInterpreter`, and on error feeds the error message back to `code_regenerate` for up to `max_iters` rounds. Once execution succeeds, `generate_output` produces the signature’s output fields. If `max_iters` is exhausted, the module raises.

dspy.CodeAct(signature, tools, max_iters=5, interpreter=None) Multiple inheritance from `ReAct` and `ProgramOfThought`. Tools must be plain `def` functions, not callable objects — the module reads `inspect.getsource(tool.func)` and injects each definition into the sandbox at the start of every `forward`. Each iteration: an inner `codeact` predictor produces Python plus a `finished` boolean; the interpreter runs the code; the trajectory dict gains a `generated_code_i` and `code_output_i` (or `observation_i` on parse/execution error). The loop exits when the LM sets `finished=True` or `max_iters` is reached. A `ChainOfThought` extractor then reads the trajectory and produces the declared outputs.

dspy.RLM(signature, max_iterations=20, max_llm_calls=50, max_output_chars=10_000, verbose=False, tools=None, sub_lm=None, interpreter=None) Experimental. A REPL-style code agent that exposes two built-in tools — `llm_query` and `llm_query_batched` — so generated code can call a separate `sub_lm` mid-execution. A shared counter across iterations enforces `max_llm_calls`; tool names are validated as Python identifiers; `SandboxSerializable` inputs encode into the sandbox so large contexts don’t have to be re-marshalled each turn. If the loop ends without an explicit submission, the extractor pass produces the final outputs from the trajectory.

Running modules in parallel

dspy.Parallel(num_threads=None, max_errors=None, access_examples=True, return_failed_examples=False, provide_traceback=None, disable_progress_bar=False, timeout=120, straggler_limit=3) Wraps `ParallelExecutor` and submits each `(module, example)` pair to a thread pool. The example can be a `dspy.Example` (unpacked via `.inputs()` when `access_examples=True`), a `dict` (unpacked as `kwargs`), a tuple (unpacked positionally), or a list (passed through when the module is itself a `Parallel`). The executor snapshots the parent’s `thread_local_overrides` and re-applies it inside each worker, so a surrounding `dspy.context(...)` is honored. Returns predictions in input order; with `return_failed_examples=True`, returns a `(results, failed_examples, exceptions)` tuple.

Related modules covered elsewhere

dspy.KNN is a retrieval helper, not a generation module — see the Retrievers reference page.

dspy.ReAct is the canonical tool-using loop and has its own page: [Tools, ReAct, and MCP](#). The wrapping machinery there is what `CodeAct` and `RLM` reuse.

Cross-links

- [Modules: composing your own](#) — every variant here is a `dspy.Module` (except `Parallel` and `majority`), so the composition rules apply.

- [Tools, ReAct, and MCP](#) — `CodeAct` and `RLM` use the same tool-wrapping machinery as `ReAct`.

- [RLM: exploring large contexts with code](#) — the deep dive on the experimental REPL-driven module summarized above.

- [Settings and context\(\)](#) — how `Parallel` and `Module.batch` snapshot the active overrides into each worker.